

INFOSYS SPRINGBOARD VIRTUAL INTERNSHIP
PROJECT COMPLETION REPORT

Multi-Agent Construction Negotiation Simulator

*A Generative-AI-Driven Framework for Simulating Multi-
Stakeholder Construction Negotiations*

Table of Contents

Table of Contents.....	2
1. Executive Summary.....	3
2. System Architecture.....	3
2.1 Frontend Layer	4
2.2 API Interface Layer	4
2.3 Core Backend Engines.....	4
2.4 Cloud LLM Acceleration	5
2.5 Technology Stack	5
2.6 Design Principle: Hybrid Intelligence	5
3. Implementation Milestones.....	5
Milestone 1 — Foundation & Configuration.....	6
Milestone 2 — AI Reasoning & Orchestration	6
Milestone 3 — Interface & Deadlock Handling	6
Milestone 4 — Outcomes & Finalisation	6
3.1 Scenario Library Delivered	7
4. Core Module Deep Dives	7
4.1 Scenario Selection and Agent Configuration Module.....	7
Agent Roster Example — Material Shortage Scenario	8
4.2 Orchestrator Agent and Negotiation Turn Management (Stateless REST API)	8
4.3 LLM-Powered Agent Reasoning and Offer Generation Engine.....	9
4.4 Counteroffer Evaluation and Concession Decision Module	10
4.4.1 Scoring Methodology	10
4.4.2 Decision Outcomes	10
4.5 Deadlock Detection and Resolution Module	11
4.5.1 Detection Algorithm.....	11
4.5.2 Resolution Pathway	12
4.5.3 Interaction with Round Limits.....	12
4.6 Negotiation Arena UI	13
4.7 Human Participant Interface.....	13
4.8 Outcome Evaluation and Report Generation	13
5. Conclusion.....	14

1. Executive Summary

Negotiation is a core operational function within the construction industry, governing procurement, subcontracting, budget reconciliation, and schedule recovery. Despite its importance, the industry lacks scalable, repeatable, and low-risk environments in which stakeholders — clients, contractors, suppliers, and finance managers — can rehearse negotiation strategy before it is applied to real commercial disputes. The Multi-Agent Construction Negotiation Simulator, branded ConstructAI, addresses this gap by using autonomous Large Language Model (LLM) agents to represent distinct construction stakeholders, each configured with an individual goal, a set of hard constraints, and a negotiation personality (Aggressive, Collaborative, or Risk-Averse).

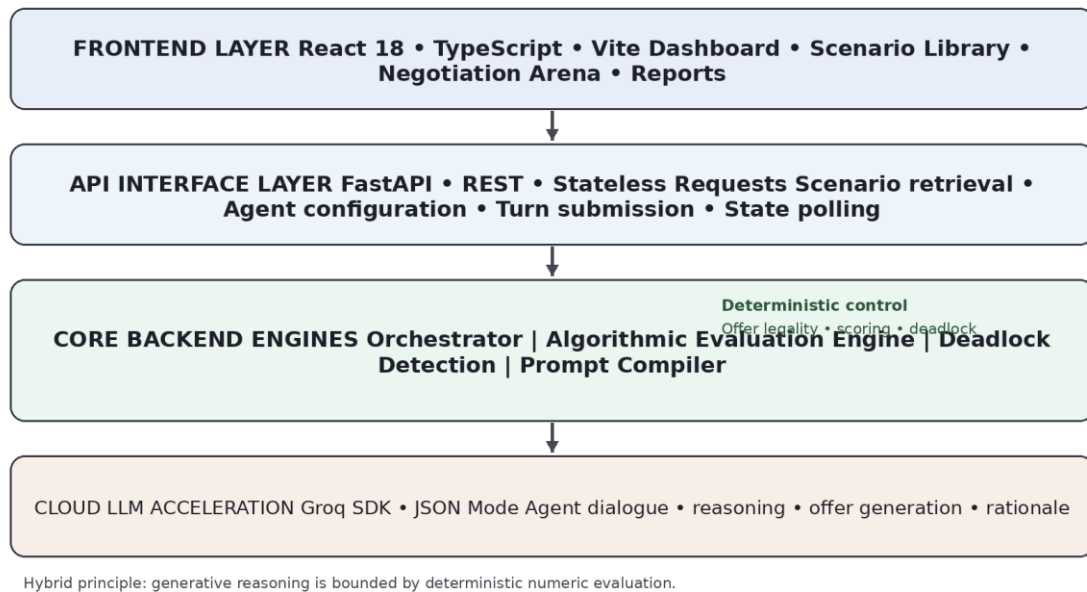
The platform is built on a hybrid-intelligence architecture that pairs generative reasoning with deterministic mathematical control. Agent dialogue, offer framing, and justification are produced through Groq-accelerated LLM inference, while offer legality, concession scoring, and deadlock detection are computed through auditable, non-probabilistic algorithms. This separation ensures that every agreement reached inside the simulator is both narratively coherent and numerically defensible.

The system supports two operating modes — Simulation Mode, in which all agents negotiate autonomously (AI vs. AI), and Practice Mode, in which a human participant negotiates directly against AI-driven counterparts (Human vs. AI) — across seven pre-built construction scenarios spanning supply chain, financial, workforce, timeline, scope, external, and equipment categories. Each completed session produces a structured outcome report, including the final agreed terms, the number of negotiation rounds elapsed, and a per-agent objective-satisfaction score, which can be exported as a transcript or summary document.

This report documents the system architecture, the four implementation milestones through which the project was delivered, and a deep technical review of the eight core modules that constitute the simulator, with particular emphasis on the Deadlock Detection Module and the Algorithmic Evaluation Engine, which together provide the deterministic backbone that keeps generative agent behaviour bounded, fair, and reproducible.

2. System Architecture

ConstructAI is organised as a hybrid intelligence platform: a layer of generative reasoning that produces agent dialogue, and a layer of strict mathematical constraint that prevents that reasoning from drifting outside a scenario's defined economics. The platform is structured into four cooperating architectural layers, described below.

Figure: ConstructAI Hybrid-Intelligence Architecture**Figure 2.1 — ConstructAI hybrid-intelligence architecture.**

2.1 Frontend Layer

The frontend is a reactive Single Page Application (SPA) responsible for the Dashboard, the Scenario Library, the Agent Configuration screen, the Negotiation Arena, and the Reports & Analytics module. It surfaces the Negotiation Arena, per-agent stance panels showing opening and current offers, concession-progress indicators, and a Human-in-the-Loop action tray used in Practice Mode. The interface follows a glassmorphism visual language — translucent panels, soft elevation, and a restrained indigo-and-navy palette — to present dense negotiation telemetry (turn counts, settlement odds, offer trajectories) without overwhelming the user.

2.2 API Interface Layer

A high-performance, asynchronous REST API mediates every interaction between the frontend and the backend engines. It exposes endpoints for scenario retrieval, agent configuration, negotiation-turn submission, live state polling, and outcome retrieval. Because the negotiation orchestrator is designed to be stateless between requests, the API layer is responsible for passing the complete negotiation history and current round context on every call, allowing the backend to remain horizontally scalable and simplifying recovery from a dropped connection mid-session.

2.3 Core Backend Engines

The backend hosts four cooperating engines: the Orchestrator, which manages turn order and round progression; the Algorithmic Evaluation Engine, which scores every offer against scenario constraints; the Deadlock Detection Engine, which monitors numeric convergence across rounds; and the Prompt

Compiler, which assembles each agent's persona, goals, constraints, and conversation history into a structured prompt for the LLM Reasoning Engine.

2.4 Cloud LLM Acceleration

Agent dialogue, reasoning, and offer justification are generated through the Groq SDK using JSON Mode, which constrains the model's output to a structured schema (offer value, offer type, rationale, and stance) rather than free-form text. This keeps generative latency low enough to sustain a responsive, chat-style negotiation transcript while guaranteeing that every LLM response can be deterministically parsed by the downstream Evaluation Engine.

2.5 Technology Stack

Layer	Technologies & Dependencies	Purpose
Backend Runtime	Python 3.10+, FastAPI, Uvicorn, Gunicorn	High-performance asynchronous REST API server handling negotiation session state.
Data Validation	Pydantic v2	Strict request/response schema validation and serialization for agent payloads.
LLM Acceleration	Groq SDK, JSON Mode	Low-latency generative inference for agent dialogue, reasoning, and offer generation.
Frontend	React 18, TypeScript, Vite	Strongly typed, reactive Single Page Application with a glassmorphism visual system.

Table 2.1 — Summary of the technology stack by architectural layer.

2.6 Design Principle: Hybrid Intelligence

The defining architectural decision in ConstructAI is the deliberate separation of creative and deterministic responsibility. The LLM Reasoning Engine is solely responsible for producing plausible, persona-consistent dialogue and offers; it is never trusted to determine whether an offer is acceptable, whether a negotiation has stalled, or what an agent's final concession should be. Those decisions are delegated to independent, non-LLM algorithms operating on the numeric content of each offer. This design keeps the simulator's outcomes auditable and reproducible even though the surface-level conversation is generated non-deterministically.

3. Implementation Milestones

The project was delivered across four sequential milestones, each building the platform from foundational configuration through to full outcome reporting.

Milestone 1 — Foundation & Configuration

- Studied negotiation theory, multi-agent system design patterns, and LLM-based agent reasoning as the theoretical basis for the simulator.
- Designed the system architecture, orchestration flow, agent persona model, and UI wireframes for all core screens.
- Implemented the Scenario Selection module with three initial pre-built templates, defining the agents, roles, goals, and hard constraints associated with each scenario.
- Built the Agent Configuration screen, featuring per-agent role cards with personality selection (Aggressive, Collaborative, Risk-Averse) and goal/constraint displays.

Milestone 2 — AI Reasoning & Orchestration

- Developed the Orchestrator Agent to manage turn order, pass negotiation history between agents, and track round count and negotiation state.
- Implemented the LLM-Powered Agent Reasoning Engine, in which each agent receives its persona, goals, constraints, and the full conversation history to generate contextually reasoned offers.
- Built the Counteroffer Evaluation and Concession Module so that agents can evaluate incoming offers against their objectives and algorithmically decide to accept, counter, or reject.
- Tested the end-to-end AI-versus-AI negotiation loop across all scenario templates to confirm stable convergence behaviour.

Milestone 3 — Interface & Deadlock Handling

- Built the Negotiation Arena UI, featuring a chat-style transcript with per-turn agent reasoning displays, live stance indicators, and a round-metrics panel.
- Implemented the Human Participant Interface for Practice Mode, allowing a user to type offers and responses while AI agents react intelligently in real time.
- Developed the Deadlock Detection Module to identify stalled negotiations after N rounds and trigger a resolution step or declare a formal breakdown.
- Validated Practice Mode across all scenarios to confirm realistic AI responses to human-authored offers.

Milestone 4 — Outcomes & Finalisation

- Built the Outcome Screen to display final agreement terms, concession timelines, rounds elapsed, and per-agent objective-satisfaction scores.
- Implemented a downloadable negotiation transcript and summary report feature, accessible per completed session and in bulk (Export All as PDF).
- Conducted end-to-end testing across both operating modes, all seven scenarios, and all personality combinations.

- Prepared technical documentation, this project report, and the final demonstration build.

3.1 Scenario Library Delivered

By the close of Milestone 4, the Scenario Library had grown from the three initial templates to seven fully configured construction negotiation scenarios, summarised in Table 3.1.

Scenario	Category	Negotiation Problem
Material Shortage	Supply Chain	Steel supply disruption; agents negotiate alternative procurement strategies, pricing, and delivery timelines.
Budget Overrun	Financial	An 18% budget overrun requires negotiation of cost-cutting measures, scope adjustments, and revised financial plans.
Labor Shortage	Workforce	A 40% shortfall in skilled labor requires negotiation of workforce solutions, overtime policy, and schedule adjustments.
Deadline Reduction	Timeline	A client-mandated 4-week schedule compression requires negotiation of resource acceleration and overtime budgets.
Scope Changes	Scope	Client-requested amenity additions require negotiation of cost, schedule, and resource impact.
Weather Delays	External	A 12-day monsoon stoppage requires negotiation of recovery plans, idle-resource costs, and revised milestones.
Equipment Breakdown	Equipment	Critical equipment failure requires negotiation of replacement, rental, and schedule-recovery terms.

Table 3.1 — The seven pre-built negotiation scenarios available in the Scenario Library, each configured for three agents across approximately ten rounds.

4. Core Module Deep Dives

This section provides a detailed technical account of the eight modules that constitute the ConstructAI simulator, in the order in which a negotiation session passes through them.

4.1 Scenario Selection and Agent Configuration Module

The Scenario Library presents each scenario as a card annotated with a difficulty rating (Easy, Medium, Hard), a category tag (Supply Chain, Financial, Workforce, Timeline, Scope, External, Equipment), the number of agents involved, and the expected round count. Selecting a scenario — for example, Material Shortage — routes the user to the Agent Configuration screen, where every participating agent is presented as a role card containing three data structures:

- **Goal** — a single natural-language objective statement, such as the Supplier Agent's mandate to maximise profit margin on steel supply while retaining the client relationship.

- **Hard Constraints** — an itemised, immutable list of numeric or categorical limits (for example, a maximum supply capacity of 600 tons, a minimum price of ₹52,000 per ton, and a fastest delivery window of five business days) that the agent must never violate regardless of negotiation pressure.
- **Negotiation Personality** — a mutually exclusive selection of Aggressive, Collaborative, or Risk-Averse, which is compiled into the agent's system prompt and directly influences its concession rate inside the Evaluation Engine.

A negotiation cannot be launched until a personality has been assigned to every agent in the scenario; the Start Negotiation control remains disabled until this condition is satisfied, preventing the orchestrator from ever receiving an under-specified agent.

Agent Roster Example — Material Shortage Scenario

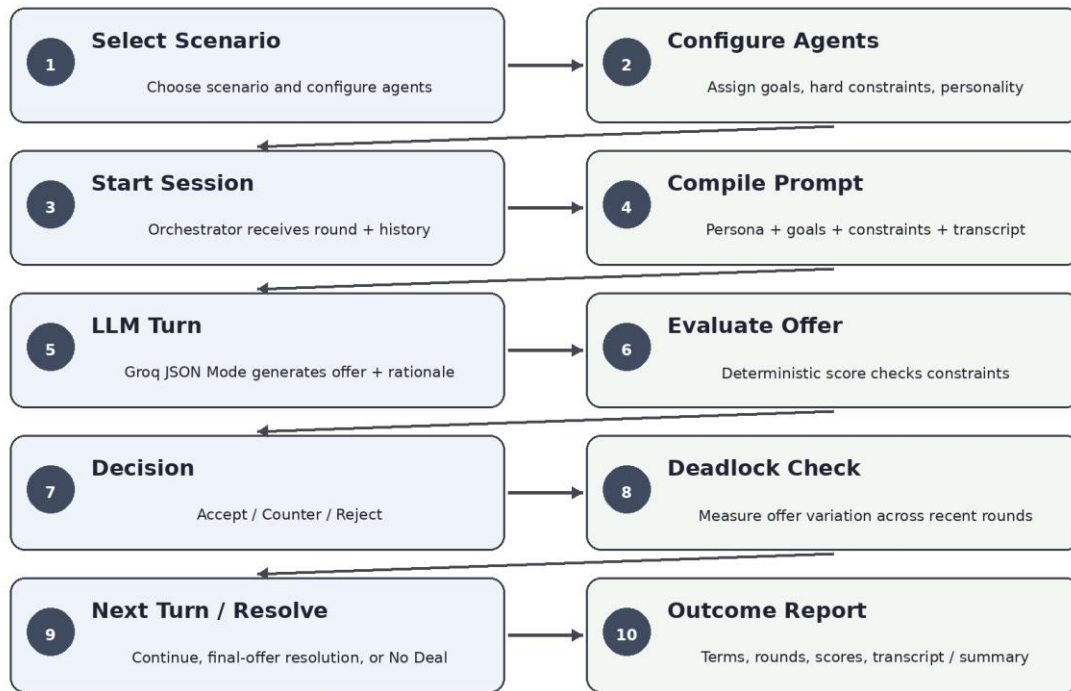
Agent	Role	Primary Goal
Supplier Agent	Material Provider	Maximise profit margin on steel supply while retaining the client relationship.
Contractor Agent	Material Buyer	Procure 600 tons of steel at the lowest possible cost with the fastest delivery.
Finance Manager Agent	Budget Gatekeeper	Keep total procurement cost within the approved project budget.
Client Agent	Client	Balance budget, scope, and deadline expectations across the project.
Project Manager Agent	Project Manager	Coordinate resources, schedule, and risk across all active negotiations.

Table 4.1 — Representative agent roster, roles, and goals for the Material Shortage scenario.

4.2 Orchestrator Agent and Negotiation Turn Management (Stateless REST API)

The Orchestrator is the coordination backbone of every negotiation session. Rather than holding session state in server memory, it is implemented as a stateless component behind the REST API: each incoming request carries the complete negotiation history, the round index, and the current stance of every agent, and the Orchestrator's sole responsibility is to compute the next valid state transition from that payload.

- Determine whose turn it is, based on a configurable turn-order policy for the active scenario.
- Assemble the negotiation history into the context window passed to the LLM Reasoning Engine for the acting agent.
- Invoke the Counteroffer Evaluation Module on any newly submitted offer and update each agent's live stance (Opening, Current Offer, Concession Progress).

Figure: End-to-End Negotiation Flow

Decision logic is deterministic; LLM output supplies dialogue, rationale, and offer framing.

Figure 4.1 — End-to-end negotiation turn and outcome flow.

- Increment the round counter and pass control to the Deadlock Detection Module before yielding the turn to the next agent.

This stateless design has two practical benefits for a generative-AI system. First, it allows the API layer to scale horizontally without sticky sessions, since any server instance can resume a negotiation purely from the payload it receives. Second, it makes every negotiation fully reproducible for audit purposes: replaying the same ordered sequence of requests against the Orchestrator always yields the same sequence of state transitions, even though the underlying LLM text generation is non-deterministic.

4.3 LLM-Powered Agent Reasoning and Offer Generation Engine

When it is an agent's turn to act, the Prompt Compiler constructs a structured prompt containing the agent's persona and personality, its goal and hard constraints, the full negotiation transcript to date, and the most recent incoming offer (if any). This prompt is submitted to the Groq-accelerated LLM in JSON Mode, which constrains the model to return a fixed schema: a numeric offer value, an offer type (opening, counter, concession, acceptance, or rejection), a short natural-language rationale for the Negotiation Arena transcript, and a self-reported stance label.

Constraining the model to JSON Mode is a deliberate reliability decision: it removes the need for brittle free-text parsing and guarantees that the numeric offer value produced by the LLM can be handed directly to the deterministic Evaluation Engine described in Section 4.4. The rationale field is retained purely for transcript readability and is never used as an input to any scoring computation.

4.4 Counteroffer Evaluation and Concession Decision Module

This module functions as an objective, mathematical counterweight to the creativity of the LLM Reasoning Engine. Every offer — whether authored by an AI agent or a human participant in Practice Mode — is passed through the Algorithmic Evaluation Engine, which computes an Offer Score on a 0–100 scale without relying on any LLM inference, thereby removing generative non-determinism from the accept/counter/reject decision.

4.4.1 Scoring Methodology

Figure: Counteroffer Evaluation & Decision Flow

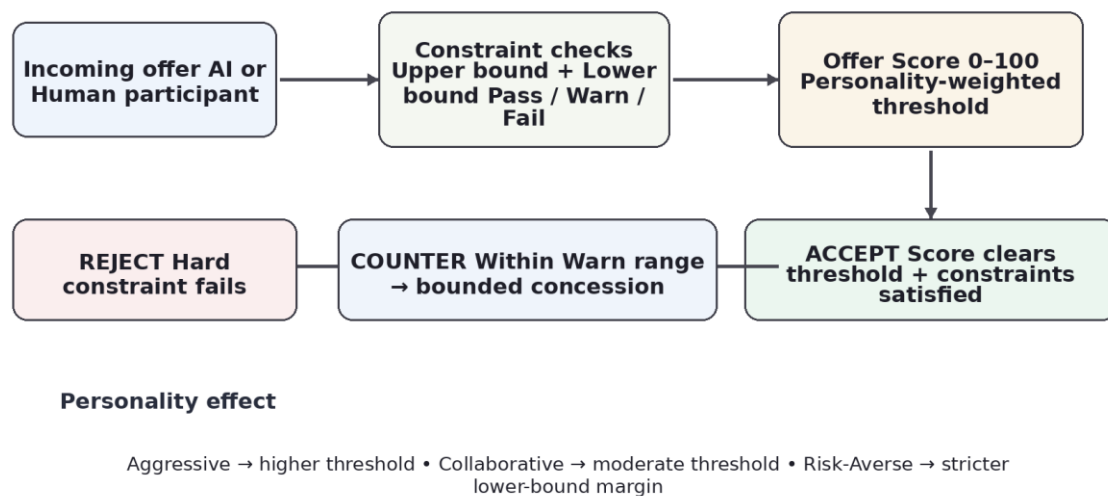


Figure 4.2 — Deterministic counteroffer evaluation and decision flow.

The Offer Score is derived from two constraint checks defined for each agent in the Agent Configuration screen:

- **Upper Bound Constraint** — evaluates the offer's proximity to the agent's maximum budget or capacity ceiling, returning a Pass (1.0) when the offer is comfortably within range, a Warn (0.5) when it approaches the ceiling, and a Fail (0.0) when it exceeds it.
- **Lower Bound Constraint** — evaluates the offer against the agent's floor price or minimum acceptable term, applying a 5% warning buffer before the offer is scored as a Fail.

The combined result of these checks is normalised into the 0–100 Offer Score, which is then interpreted through a personality-weighted decision threshold: an Aggressive agent requires a materially higher score before conceding, a Collaborative agent accepts a moderate score more readily, and a Risk-Averse agent applies a stricter lower-bound margin before it will counter rather than reject outright. This is the mechanism through which the qualitative personality selected in Agent Configuration is translated into quantitative negotiation behaviour.

4.4.2 Decision Outcomes

- **Accept** — the incoming offer's score clears the agent's personality-weighted threshold and satisfies both bound constraints.
- **Counter** — the offer fails the threshold but remains within the Warn range; the module computes a bounded concession step toward the midpoint of the negotiation range.
- **Reject** — the offer fails a hard constraint outright (for example, a price below the agent's non-negotiable floor), and the agent is instructed to hold position or exit.

4.5 Deadlock Detection and Resolution Module

Because negotiating agents can, in principle, exchange offers indefinitely without converging, the simulator implements a dedicated Deadlock Detection Module that runs after every round to protect the session from looping without bound. This module is deliberately implemented as pure numeric analysis, entirely independent of the LLM, so that a stalled negotiation can always be detected even if the language model itself fails to recognise repetition in its own output.

4.5.1 Detection Algorithm

The algorithm maintains a rolling window of the last N rounds of numeric proposals for the negotiation (N being a configurable parameter, defaulted per scenario). At the close of each round it:

Figure: Deadlock Detection & Resolution Pathway

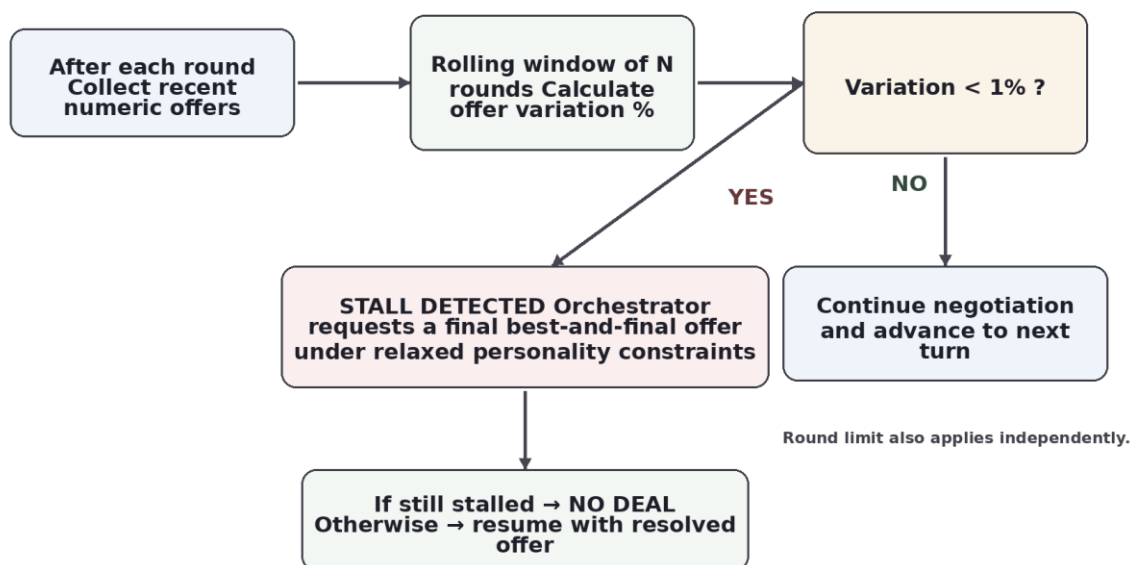


Figure 4.3 — Deadlock detection, resolution, and termination pathway.

- Extracts the numeric offer value submitted by each active agent across the monitored window.
- Calculates the variation percentage between the earliest and most recent offers in that window, relative to the offer magnitude.
- Flags the negotiation as stalled the moment this variation percentage falls below a 1% threshold, indicating that successive offers are no longer moving meaningfully toward a settlement.

This variance-based approach was chosen over a simple repeated-value check because real negotiation offers rarely repeat exactly — agents typically make token, cosmetic concessions (for instance, a fractional per-ton price change) that would defeat an equality-based test while still representing an effectively frozen negotiation. Measuring percentage variation across a rolling window correctly classifies this behaviour as deadlock.

Figure: Negotiation Arena UI Layout

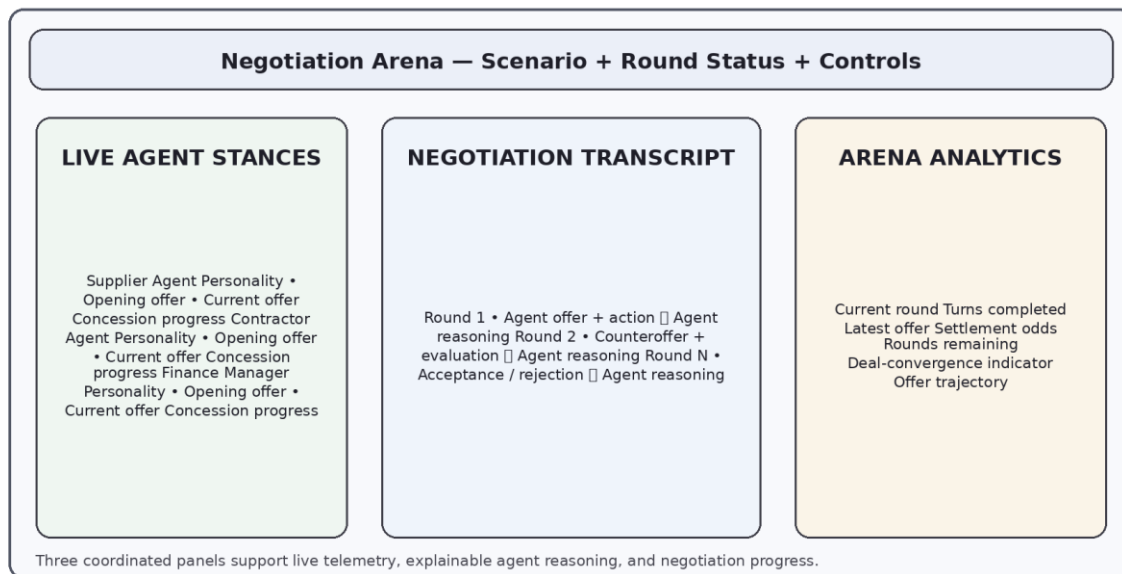


Figure 4.4 — Conceptual layout of the Negotiation Arena UI.

4.5.2 Resolution Pathway

Once a stall is flagged, the module does not immediately terminate the session. It first triggers a resolution step in which the Orchestrator instructs the involved agents to submit a final, best-and-final offer under relaxed personality constraints, giving the negotiation one last opportunity to converge. If the subsequent round still fails to clear the 1% variance threshold, the negotiation is formally declared a breakdown (No Deal) and routed to the Outcome Screen with that status, rather than being allowed to consume further rounds without purpose. This two-stage design — detect, then attempt resolution, then declare breakdown — mirrors real-world negotiation practice, in which a stalemate is typically met with a final offer before parties walk away.

4.5.3 Interaction with Round Limits

The Deadlock Detection Module operates independently of, but alongside, the scenario's overall round limit (approximately ten rounds for the current scenario library). A negotiation may therefore end for either of two reasons: exhaustion of the configured round budget, or an explicit deadlock declaration issued by this module — both of which are recorded distinctly in the Reports & Analytics log so that the two failure modes can be distinguished during post-session analysis.

4.6 Negotiation Arena UI

The Negotiation Arena is the primary runtime interface for an active session. It is organised into three coordinated panels: a Live Agent Stances panel on the left, listing every participating agent with its personality tag, opening offer, current offer, and a concession-progress bar; a central Negotiation Transcript, which streams a chat-style log of each agent's turn together with its generated rationale once the simulation begins; and an Arena Analytics panel on the right, surfacing settlement odds, the latest offer, turns elapsed, rounds remaining, a deal-convergence indicator, and an offer-trajectory chart that renders as agents submit successive offers.

Before a session starts, the Arena presents an idle state ("Arena is Ready for Negotiation") confirming that all agents have been initialised with their configured constraints and goals, and exposes two explicit entry points corresponding to the platform's two operating modes: AI Simulation and Practice Mode.

4.7 Human Participant Interface

Practice Mode activates the Human Participant Interface, through which a user assumes the role of one negotiating stakeholder and submits offers and responses directly, while the remaining seats are filled by AI agents that continue to reason and respond in real time using the same LLM Reasoning Engine and Algorithmic Evaluation Engine described in Sections 4.3 and 4.4. Because the human's offers are numeric inputs routed through the identical Evaluation Engine used for AI-authored offers, the human participant is held to exactly the same deterministic accept/counter/reject logic as an AI agent, ensuring that Practice Mode sessions are a fair rehearsal of the mechanics a real negotiator would face rather than a scripted or artificially lenient experience.

4.8 Outcome Evaluation and Report Generation

On completion of a session — whether by agreement, round exhaustion, or a declared deadlock — the platform compiles an Outcome Report summarising the final agreed terms (or the last offer on the table if no agreement was reached), the total number of rounds elapsed, the concession timeline for each agent, and a per-agent objective-satisfaction score derived from the same Algorithmic Evaluation Engine used during the live negotiation. Every completed session is archived in the Reports & Analytics module, where it can be reviewed alongside aggregate platform metrics — total negotiations run, the overall success rate, the average rounds per session, and the estimated cost optimisation achieved across agreed sessions — and exported per-session as a Transcript or Summary document, or in bulk as a consolidated PDF.

Scenario	Category	Agents	Rounds	Final Offer	Outcome
Material Shortage	Supply Chain	3	4	₹52,000/ton	Agreed
Labor Shortage	Workforce	3	3	1.5x rate	Agreed
Budget Overrun	Financial	3	5	—	No Deal

Table 4.2 — Representative Negotiation Performance Log entries recorded by the Reports & Analytics module.

The three recorded sessions illustrate the range of terminal states the platform supports: the Material Shortage and Labor Shortage scenarios each converged to an agreement within three to four rounds, while the Budget Overrun scenario exhausted five rounds without clearing the Deadlock Detection Module's convergence threshold and was correctly logged as No Deal — evidence that the deterministic evaluation and deadlock layers behave as designed under genuinely constrained negotiating positions, rather than forcing every session toward an artificial settlement.

5. Conclusion

The Multi-Agent Construction Negotiation Simulator demonstrates that generative AI can be deployed responsibly within a domain — commercial negotiation — that ordinarily demands numerical precision and auditability, by strictly separating what the language model is asked to do from what it is trusted to decide. The LLM Reasoning Engine supplies persona-consistent dialogue and plausible offer framing; the Algorithmic Evaluation Engine and Deadlock Detection Module supply the deterministic, reproducible judgement of whether that dialogue represents a legitimate, converging, or stalled negotiation. This hybrid-intelligence pattern proved effective across all seven scenario templates and both operating modes, producing sessions that were narratively coherent, numerically consistent with each agent's configured constraints, and reliably terminated — whether in agreement or in a correctly flagged breakdown.

Across the four implementation milestones, the project progressed from foundational scenario and agent modelling, through orchestration and LLM reasoning, to a fully interactive Negotiation Arena with human participation and deadlock handling, culminating in a structured outcome-reporting layer. The resulting platform offers a scalable, repeatable, and risk-free environment in which construction stakeholders can rehearse negotiation strategy — fulfilling the objective set out at the start of this project — and establishes an architectural pattern (generative reasoning bounded by deterministic evaluation) that is readily extensible to additional negotiation domains beyond construction procurement.